TF4015 – Sistem Kontrol Diskrit

Praktikum

Modul 4

Pole Placement
dan Observer

Tim Pengajar TF4015

Program Studi Teknik Fisika
Fakultas Teknologi Industri
Institut Teknologi Bandung

Semester I 2026/2027

Daftar Isi

1 Pendahuluan **1**
1.1 Latar Belakang . . . 1
1.2 Tujuan Praktikum . . . 1
1.3 Peralatan dan Perangkat Lunak . . . 2
1.4 Persiapan Praktikum . . . 2

2 Teori Dasar **2**
2.1 Model Ruang Keadaan Kit Praktikum . . . 2
2.2 Keterkendalian dan Keteramatan . . . 3
2.3 Pole Placement . . . 3
2.4 Observer . . . 4
2.5 Prinsip Separasi . . . 5
2.6 Implementasi pada Program . . . 5

3 Keselamatan Kerja **5**

4 Prosedur Praktikum **5**
4.1 Percobaan 1: Model Ruang Keadaan dan Desain . . . 5
4.2 Percobaan 2: Pole Placement Kecepatan . . . 6
4.3 Percobaan 3: Uji Observer pada Kalang Terbuka . . . 7
4.4 Percobaan 4: Pole Placement Posisi dengan Observer . . . 7
4.5 Percobaan 5: Perbandingan dengan Simulasi . . . 8

5 Laporan dan Analisis **8**
5.1 Ketentuan Laporan . . . 8
5.2 Analisis . . . 9

6 Tugas Tambahan **10**

A Kode Program Pendukung **11**

1 Pendahuluan

1.1 Latar Belakang

Pada Modul 2 dan Modul 3, pengontrol PID ditala dengan rumus empirik lalu diperbaiki dengan penalaan halus. Cara ini praktis, tetapi letak kutub sistem kalang tertutup tidak ditentukan secara langsung. Pada **umpan balik keadaan** (*state feedback*), seluruh keadaan sistem diumpanbalikkan dengan gain yang dipilih sehingga kutub kalang tertutup berada di lokasi yang diinginkan. Metode ini disebut **penempatan kutub** (*pole placement*).

Umpan balik keadaan membutuhkan seluruh keadaan sistem. Pada kit praktikum, keadaan sistem adalah posisi dan kecepatan. Keduanya memang diukur, tetapi masing-masing sensor memiliki keterbatasan. Tachometer tidak membedakan arah putar, membaca nol pada kecepatan rendah, dan memiliki filter dengan konstanta waktu cukup besar. Potensiometer hanya memberikan posisi, dan turunan numeriknya sangat berderau. **Observer** mengatasi masalah ini dengan memperkirakan keadaan menggunakan model sistem yang dikoreksi oleh pengukuran.

Pada Modul 4, praktikan menyusun model ruang keadaan diskrit dari hasil Modul 1 dan Modul 3, menerapkan pole placement untuk kontrol kecepatan dan posisi, serta merancang dua observer: observer yang memperkirakan **kecepatan dari potensiometer** dan estimator yang memperkirakan **posisi dari tachometer**. Kedua observer dibandingkan, dan konsep keteramatan (*observability*) akan menjelaskan mengapa hasil keduanya sangat berbeda.

1.2 Tujuan Praktikum

Setelah menyelesaikan Modul 4, praktikan diharapkan mampu:

1. Menyusun model ruang keadaan diskrit kit praktikum serta menguji keterkendalian dan keteramatannya.
2. Merancang dan mengimplementasikan pole placement untuk kontrol kecepatan dan kontrol posisi.
3. Merancang dan mengimplementasikan observer untuk mengestimasi kecepatan menggunakan potensiometer dan mengestimasi posisi menggunakan tachometer.
4. Mengimplementasikan kontrol posisi berbasis observer dan membandingkannya dengan umpan balik sensor secara langsung.

1.3 Peralatan dan Perangkat Lunak

Tabel 1: Daftar peralatan praktikum Modul 4

No	Nama Peralatan	Keterangan
1	Kit praktikum SKD (kit yang sama dengan modul sebelumnya)	Disediakan asisten
2	Laptop dengan Arduino IDE dan *library* Adafruit MCP4725	Milik praktikan
3	MATLAB (desktop atau MATLAB Online)	Milik praktikan
4	Perangkat lunak *spreadsheet*	Milik praktikan
5	Program: tab baru `6_state_space.ino` dan program utama `11_main_state_space`	Unduh dari repositori
6	Skrip MATLAB `desain_state_space.m` dan `analisis_observer.m`	Unduh dari repositori
7	Model Modul 1 (K, τ, θ), faktor konversi C (Modul 3), dan pengontrol posisi terbaik (Modul 3)	Hasil praktikum sebelumnya

1.4 Persiapan Praktikum

1. Siapkan parameter model K, τ, θ (Modul 1) dan faktor konversi C hasil eksperimen (Modul 3).
2. Tambahkan tab `6_state_space.ino` (Kode 1) ke folder program. Pada Arduino IDE, tab baru dapat dibuat melalui menu panah di sebelah kanan deretan tab.
3. Jalankan skrip `desain_state_space.m` (Kode 3) dengan model kelompok dan pahami hasil simulasinya.
4. Baca prosedur dan bagian analisis, lalu bagi tugas dalam kelompok.

2 Teori Dasar

2.1 Model Ruang Keadaan Kit Praktikum

Dengan keadaan posisi potensiometer p (%) dan kecepatan bertanda v (%), model kecepatan FOPDT (tanpa waktu tunda) dan hubungan posisi–kecepatan dari Modul 3 dapat ditulis sebagai

$$\dot{x} = \begin{bmatrix} 0 & C \\ 0 & -1/\tau \end{bmatrix} x + \begin{bmatrix} 0 \\ K/\tau \end{bmatrix} u, \qquad x = \begin{bmatrix} p \\ v \end{bmatrix}, \tag{1}$$

dengan K dan τ dari Modul 1 serta C dari Modul 3. Dua keluaran pengukuran tersedia:

$$y_p = C_p\, x = \begin{bmatrix} 1 & 0 \end{bmatrix} x \;\; \text{(potensiometer)}, \qquad y_v = C_v\, x = \begin{bmatrix} 0 & 1 \end{bmatrix} x \;\; \text{(tachometer)}. \tag{2}$$

Dengan masukan yang ditahan konstan selama satu waktu cacah (ZOH), model diskritnya adalah

$$x[k{+}1] = A\,x[k]{+}B\,u[k], \qquad A = \begin{bmatrix} 1 & C\tau(1-a) \\ 0 & a \end{bmatrix}, \qquad B = \begin{bmatrix} CK\,(T_s - \tau(1-a)) \\ K(1-a) \end{bmatrix}, \tag{3}$$

dengan $a = e^{-T_s/\tau}$. Baris kedua persamaan ini sama dengan model diskrit kecepatan pada Modul 1, sedangkan baris pertama diperoleh dengan mengintegralkan kecepatan selama satu waktu cacah. Waktu tunda θ diabaikan pada desain, tetapi tetap disertakan dalam simulasi untuk melihat pengaruhnya.

2.2 Keterkendalian dan Keteramatan

Sistem (3) **terkendali** bila matriks keterkendalian

$$W_c = \begin{bmatrix} B & AB \end{bmatrix} \tag{4}$$

berpangkat penuh (pangkat 2). Bila terkendali, seluruh kutub kalang tertutup dapat ditempatkan sembarang dengan umpan balik keadaan.

Sistem **teramati** dari keluaran $y = C_y x$ bila matriks keteramatan

$$W_o = \begin{bmatrix} C_y \\ C_y A \end{bmatrix} \tag{5}$$

berpangkat penuh. Bila teramati, seluruh keadaan dapat diperkirakan dari keluaran tersebut dengan error yang konvergen ke nol. Praktikan akan memeriksa keteramatan untuk dua pilihan keluaran pada Persamaan (2).

2.3 Pole Placement

Kontrol kecepatan (satu keadaan). Model kecepatan $v[k+1] = a\,v[k] + b\,u[k]$, dengan $b = K(1-a)$, dikontrol dengan

$$u[k] = N\,r[k] - k_v\,v[k]. \tag{6}$$

Kutub kalang tertutupnya adalah $z_c = a - b\,k_v$. Untuk kutub yang diinginkan $z_c = e^{-T_s/\tau_c}$, dengan τ_c konstanta waktu kalang tertutup yang diinginkan,

$$k_v = \frac{a - z_c}{b}, \qquad N = \frac{1 - z_c}{b}, \tag{7}$$

dengan N dipilih agar kecepatan tunak sama dengan set point bila model tepat.

Kontrol posisi (dua keadaan). Posisi dikontrol dengan

$$u[k] = -k_1\ \mathrm{wrap}(p[k] - r) - k_2\,v[k] = -\mathbf{K}\,\tilde{x}[k], \qquad \mathbf{K} = \begin{bmatrix} k_1 & k_2 \end{bmatrix}, \tag{8}$$

dengan $\tilde{x} = [p - r \;\; v]^T$. Karena sistem posisi sudah memiliki integrator, posisi tunak sama dengan set point tanpa gain umpan maju. Gain $\mathbf{K}$ dipilih agar polinomial karakteristik $\det(zI - A + B\mathbf{K})$ sama dengan polinomial yang diinginkan $\alpha_c(z) = (z - z_1)(z - z_2)$. Untuk sistem orde dua, rumus Ackermann memberikan

$$\mathbf{K} = \begin{bmatrix} 0 & 1 \end{bmatrix} W_c^{-1}\,\alpha_c(A), \qquad \alpha_c(A) = A^2 + \alpha_1 A + \alpha_0 I. \tag{9}$$

Pemilihan kutub. Kutub diskrit dapat dipilih dari spesifikasi rasio redaman ζ dan *settling time* 2% t_s sistem kontinu yang diinginkan:

$$\omega_n = \frac{4}{\zeta\, t_s}, \qquad s_{1,2} = -\zeta\omega_n \pm j\,\omega_n\sqrt{1-\zeta^2}, \qquad z_{1,2} = e^{s_{1,2}T_s}. \tag{10}$$

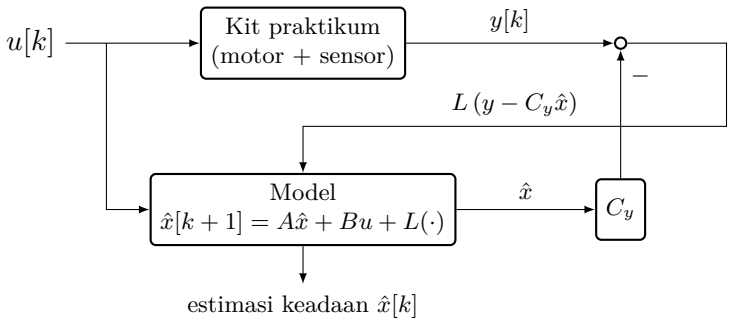
Kutub yang terlalu cepat membutuhkan aksi kontrol besar sehingga mudah saturasi, serta lebih peka terhadap waktu tunda dan derau.

2.4 Observer

Observer Luenberger memperkirakan keadaan $\hat{x}$ dengan menjalankan model secara paralel dan mengoreksinya dengan selisih keluaran terukur dan keluaran perkiraan:

$$\hat{x}[k+1] = A\,\hat{x}[k] + B\,u[k] + L\,(y[k] - C_y\,\hat{x}[k])\,. \tag{11}$$

Error estimasi $\tilde{e} = x - \hat{x}$ memenuhi $\tilde{e}[k+1] = (A - LC_y)\,\tilde{e}[k]$, sehingga gain L dipilih agar kutub $A - LC_y$ cukup cepat, umumnya 2–5 kali lebih cepat daripada kutub kontrol. Karena masalah ini adalah dual dari pole placement, L dapat dihitung dengan rumus Ackermann pada pasangan (A^T, C_y^T). Struktur observer ditunjukkan pada Gambar 1.



Gambar 1: Struktur observer Luenberger

Observer A: kecepatan dari potensiometer. Dengan $C_y = C_p = [1 \quad 0]$, observer menggunakan bacaan posisi untuk memperkirakan posisi dan kecepatan. Karena bacaan posisi berpindah dari 100% ke 0%, selisih $y - C_y\hat{x}$ dibatasi dengan fungsi wrap seperti pada Modul 3, dan $\hat{p}$ dijaga pada rentang 0–100%. Kecepatan perkiraan $\hat{v}$ memiliki tanda (arah putar) dan tetap tersedia pada kecepatan rendah ketika tachometer membaca nol.

Observer B: posisi dari tachometer. Dengan $C_y = C_v = [0 \;\; 1]$, observer menggunakan bacaan kecepatan. Tachometer tidak membedakan arah putar, sehingga kecepatan bertanda didekati dengan

$$v_m = \text{sign(arah perintah terakhir)} \times \text{bacaan tachometer}, \tag{12}$$

dengan arah diambil dari variabel `Direction` pada tab `2_actuator.ino`. Matriks $A - LC_v$ berbentuk segitiga atas dengan satu nilai eigen selalu di $z = 1$, berapa pun nilai L. Gain $L_{v1} = C\tau(1-a)$ membuat posisi perkiraan diperoleh dengan mengintegralkan kecepatan terukur (*dead reckoning*), sedangkan $L_{v2} = a - z_v$ menempatkan kutub koreksi kecepatan di z_v. Posisi awal estimator diambil dari bacaan potensiometer saat program dimulai. Praktikan akan menyelidiki akibat sifat ini melalui pemeriksaan keteramatan dan percobaan.

2.5 Prinsip Separasi

Bila umpan balik keadaan menggunakan keadaan perkiraan, $u = -\mathbf{K}\hat{x}$, kutub sistem kalang tertutup gabungan adalah gabungan kutub $A - B\mathbf{K}$ dan kutub $A - LC_y$. Dengan demikian, pengontrol dan observer dapat dirancang secara terpisah. Prinsip ini berlaku untuk model linier yang tepat; ketidaktepatan model, waktu tunda, saturasi, dan *deadzone* pada kit dapat membuat hasilnya berbeda.

2.6 Implementasi pada Program

Pada setiap waktu cacah, program `11_main_state_space` melakukan langkah berikut:

1. Membaca posisi $y_p[k]$ dan tachometer, lalu menghitung kecepatan bertanda $v_m[k]$.
2. Menghitung aksi kontrol $u[k]$ sesuai mode, menggunakan $\hat{x}[k]$ (bila observer dipakai) atau pengukuran langsung, lalu membatasinya.
3. Memberikan $u[k]$ ke motor melalui kompensasi *deadzone*.
4. Memperbarui kedua observer dengan Persamaan (11) untuk memperoleh $\hat{x}[k+1]$.

Model menggunakan masukan sebelum kompensasi u, sama seperti model Modul 1. Matriks A dan B dihitung otomatis oleh fungsi `state_space_setup()` dari `ModelK`, `ModelTau`, `ModelC`, dan waktu cacah.

3 Keselamatan Kerja

Seluruh ketentuan keselamatan kerja dan SOP pada modul sebelumnya tetap berlaku. Perhatikan pula hal-hal berikut:

1. Periksa setiap gain hasil desain dengan simulasi sebelum diterapkan pada kit, dan verifikasi program tanpa daya motor.
2. Mulailah dengan kutub kontrol yang lambat. Kutub yang terlalu cepat dapat menyebabkan osilasi keras.
3. Pada Mode 0, urutan masukan harus melewati 0 sebelum berganti arah.
4. Bila motor berosilasi keras atau berputar tidak terkendali, segera cabut kabel listrik 220 V.

4 Prosedur Praktikum

Prosedur Modul 4 terdiri atas lima percobaan. Simpan seluruh data mentah ke file teks.

4.1 Percobaan 1: Model Ruang Keadaan dan Desain

1. Isi parameter model dan waktu cacah pada skrip `desain_state_space.m`. Gunakan waktu cacah pertama dari Modul 1 atau waktu cacah lain yang dijustifikasi.

2. Jalankan skrip dan lengkapi Tabel 2 dengan matriks A dan B, pangkat matriks keterkendalian, serta pangkat matriks keteramatan untuk kedua keluaran.
3. Tentukan dua set spesifikasi kontrol posisi (Desain 1 lebih lambat, Desain 2 lebih cepat), satu set spesifikasi kontrol kecepatan, dan spesifikasi observer. Catat gain dan kutub hasil desain pada Tabel 3.
4. Amati grafik simulasi yang dihasilkan skrip. Pastikan simulasi stabil dan aksi kontrol tidak terus-menerus saturasi sebelum gain diterapkan pada kit.

Tabel 2: Model ruang keadaan diskrit kit praktikum

Besaran	**Nilai**
K, τ (ms), θ (ms), C, T_s (ms)	
Matriks A	
Matriks B	
Pangkat W_c	
Pangkat W_o dengan $C_p = [1 \;\; 0]$	
Pangkat W_o dengan $C_v = [0 \;\; 1]$	

Tabel 3: Spesifikasi, kutub, dan gain hasil desain

Desain	**Spesifikasi**	**Kutub diskrit**	**Gain**
Kecepatan	$\tau_c =$	$z_c =$	$k_v =$ $N =$
Posisi, Desain 1	$\zeta =$ $t_s =$		$k_1 =$ $k_2 =$
Posisi, Desain 2	$\zeta =$ $t_s =$		$k_1 =$ $k_2 =$
Observer A	$\zeta =$ $t_s =$		$L_{p1} =$ $L_{p2} =$
Observer B	$\tau_v =$		$L_{v1} =$ $L_{v2} =$

4.2 Percobaan 2: Pole Placement Kecepatan

1. Ganti tab `9_main.ino` dengan program `11_main_state_space` (Kode 2). Salin baris hasil skrip ke `setup()`, atur `TimeSampling` sesuai desain, `Mode = 1`, dan `Sequence = {0, 40, 70, 30, 0}`.
2. Verifikasi tanpa daya motor bahwa $U = Nr$ ketika kecepatan nol, lalu jalankan dengan daya motor dan simpan datanya.
3. Ulangi dengan konstanta waktu kalang tertutup lain (misalnya τ_c dua kali lebih kecil).
4. Untuk setiap perubahan set point, ukur konstanta waktu kalang tertutup (waktu mencapai 63,2% perubahan) dan offset. Lengkapi Tabel 4.

Tabel 4: Performa pole placement kecepatan

τ_c **desain (ms)**	**Step**	τ_c **terukur (ms)**	**Offset (%)**	u **maksimum (%)**
	$0 \rightarrow 40$			
	$40 \rightarrow 70$			
	$70 \rightarrow 30$			
	$0 \rightarrow 40$			
	$40 \rightarrow 70$			
	$70 \rightarrow 30$			

4.3 Percobaan 3: Uji Observer pada Kalang Terbuka

1. Atur `Mode = 0`. Pilih masukan kecil u_{kecil} yang membuat motor berputar pelan tetapi tachometer masih membaca nol atau hampir nol (lihat spesifikasi kit pada Modul 1). Atur `Sequence = {0,` u_{kecil}`, 0, 30, 0, -30, 0, 60, 0, -60, 0}`.
2. Verifikasi tanpa daya motor, lalu jalankan dan simpan data ke file `data_observer.txt`. Program mencetak posisi terukur, posisi perkiraan observer A (`PosHat`), posisi perkiraan observer B (`PosHatT`), kecepatan bertanda dari tachometer (`VelT`), kecepatan perkiraan observer A (`VelHat`), dan masukan.
3. Jalankan skrip `analisis_observer.m` (Kode 4) untuk membandingkan $\hat{v}$ dengan kecepatan tachometer dan dengan turunan numerik posisi, serta $\hat{p}$ observer B dengan posisi terukur.
4. Ulangi dengan observer A yang lebih lambat dan yang lebih cepat (ubah `TsObs` pada skrip desain, lalu salin gain baru). Lengkapi Tabel 5.
5. **Uji simpangan observer B**: jalankan urutan yang lebih panjang (misalnya ulangi urutan masukan dua hingga tiga kali). Catat error posisi observer B pada awal dan akhir percobaan.

Tabel 5: Hasil uji observer pada kalang terbuka

Observer A (t_s **obs.)**	**RMSE** $\hat{v}$ **vs tachometer (%)**	**RMSE turunan numerik (%)**	$\hat{v}$ **pada** u_{kecil} **(%)**	**Error** $\hat{p}_B$ **awal (%)**	**Error** $\hat{p}_B$ **akhir (%)**

4.4 Percobaan 4: Pole Placement Posisi dengan Observer

1. Atur `Mode = 2`, `UseObserver = true`, batas `UMax` sesuai Modul 3, gain Desain 1, dan `Sequence = {50, 20, 50, 90, 20, 50}`.
2. Verifikasi tanpa daya motor, lalu jalankan dan simpan datanya. Hitung performa untuk perubahan set point $50 \rightarrow 20$ dan $20 \rightarrow 50$.

3. Ulangi dengan gain Desain 2.
4. Ulangi Desain 1 dengan `UseObserver = false`, sehingga kecepatan umpan balik diambil dari tachometer bertanda.
5. Lengkapi Tabel 6. Tambahkan pengontrol posisi terbaik dari Modul 3 sebagai pembanding.
6. **Uji gangguan**: dengan desain terbaik, setelah urutan selesai set point terakhir dipertahankan. Berikan gangguan dengan memutar roda gigi potensiometer sedikit, lalu lepaskan. Simpan responsnya.

Tabel 6: Performa pole placement posisi

Pengontrol	**Step**	t_d **(ms)**	t_r **(ms)**	M_p **(%)**	t_s **(ms)**	**Offset (%)**
Desain 1, observer	$50 \rightarrow 20$					
	$20 \rightarrow 50$					
Desain 2, observer	$50 \rightarrow 20$					
	$20 \rightarrow 50$					
Desain 1, tachometer	$50 \rightarrow 20$					
	$20 \rightarrow 50$					
Terbaik Modul 3	$50 \rightarrow 20$					
	$20 \rightarrow 50$					

4.5 Percobaan 5: Perbandingan dengan Simulasi

1. Pada skrip `desain_state_space.m`, ubah urutan set point simulasi agar sama dengan percobaan, lalu bandingkan respons simulasi dengan respons kit untuk Desain 1 dan Desain 2.
2. Pada simulasi, ubah waktu tunda `Theta` menjadi nol dan dua kali nilai sebenarnya. Amati pengaruh waktu tunda terhadap respons pole placement.

5 Laporan dan Analisis

5.1 Ketentuan Laporan

Laporan dikumpulkan per kelompok melalui Edunex paling lambat dua minggu setelah praktikum. Nilai tambahan diberikan bila laporan dikumpulkan dalam satu minggu. Laporan berupa file `.pdf` yang dilampiri *spreadsheet* pengolahan data, data mentah (`.txt`), program Arduino, dan skrip MATLAB, digabung dalam satu file `.zip`. Laporan minimal memiliki susunan berikut: latar belakang, tujuan, alat, prosedur, analisis, kesimpulan, kontribusi, dan referensi.

Latar belakang, tujuan, alat, dan prosedur tidak boleh sama persis dengan modul ini. Analisis disusun dengan alur yang runtut dengan menjawab pertanyaan pada Bagian 5.2. Perhatikan penggunaan angka penting. Bagian kontribusi berisi Tabel 7.

Tabel 7: Kontribusi anggota kelompok

No	NIM	Nama Lengkap	Kontribusi
1			
2			
3			
4			
5			

5.2 Analisis

Model ruang keadaan.

1. Turunkan Persamaan (3) dari Persamaan (1). Tunjukkan bahwa baris kedua sama dengan model diskrit kecepatan pada Modul 1.
2. Apakah sistem terkendali? Apakah sistem teramati dari potensiometer? Apakah sistem teramati dari tachometer? Jelaskan arti fisis hasil tersebut.
3. Di mana letak kutub sistem kalang terbuka? Apa arti kutub di $z = 1$?

Pole placement kecepatan.

4. Bandingkan konstanta waktu kalang tertutup terukur dengan desain. Bila berbeda, apa penyebabnya?
5. Apakah terdapat offset? Jelaskan penyebabnya dan usulkan cara menghilangkannya tanpa mengubah kutub yang dirancang.
6. Bandingkan pole placement kecepatan dengan pengontrol terbaik Modul 2 dari segi kecepatan respons, offset, dan kemudahan perancangan.

Observer.

7. Bandingkan $\hat{v}$ observer A dengan kecepatan tachometer dan dengan turunan numerik posisi. Mana yang paling halus, dan mana yang paling akurat? Jelaskan.
8. Apa yang terlihat pada $\hat{v}$ ketika masukan u_{kecil} diberikan? Apa keunggulan observer A dibandingkan tachometer pada kondisi ini dan saat motor berbalik arah?
9. Bagaimana pengaruh kecepatan kutub observer A terhadap derau dan keterlambatan estimasi? Bagaimana memilih kutub observer yang baik?
10. Apakah terdapat selisih tetap antara $\hat{p}$ observer A dan posisi terukur ketika motor berputar dengan kecepatan konstan? Kaitkan dengan waktu tunda yang diabaikan pada model.
11. Tunjukkan error posisi observer B terhadap waktu. Apakah error konvergen ke nol? Jelaskan hasilnya berdasarkan pemeriksaan keteramatan dan nilai eigen $A - LC_v$.
12. Sebutkan sumber-sumber error estimasi posisi dari tachometer (misalnya tanda kecepatan, kecepatan rendah, filter tachometer, dan ketidaktepatan C). Mana yang paling dominan pada percobaan?

13. Bagaimana estimasi posisi dari tachometer dapat diperbaiki bila potensiometer tetap tersedia? Kaitkan jawaban dengan observer A.

Pole placement posisi dengan observer.

14. Bandingkan respons Desain 1 dan Desain 2 dengan spesifikasi desain dan dengan simulasi. Apakah kutub yang lebih cepat selalu menghasilkan respons yang lebih baik pada kit?
15. Bandingkan umpan balik kecepatan dari observer dan dari tachometer bertanda. Mana yang lebih baik, dan mengapa?
16. Bandingkan pole placement posisi dengan pengontrol terbaik Modul 3. Kaitkan struktur $u = -k_1 e - k_2 v$ dengan pengontrol PD.
17. Bagaimana respons terhadap gangguan manual? Apakah observer tetap memberikan estimasi yang baik selama gangguan?
18. Berdasarkan simulasi dengan waktu tunda yang berbeda, seberapa besar pengaruh waktu tunda yang diabaikan pada desain?

6 Tugas Tambahan

Tugas tambahan bersifat opsional dan dapat ditambahkan pada laporan untuk nilai tambah.

1. Tambahkan keadaan integral error, $x_I[k+1] = x_I[k] + (r - v[k])$, pada kontrol kecepatan, lalu rancang pole placement untuk sistem orde dua hasil augmentasi. Apakah offset hilang?
2. Masukkan waktu tunda ke dalam model dengan menambahkan keadaan $u[k-1], \ldots, u[k-d]$. Rancang ulang pengontrol dan observer, lalu bandingkan hasilnya.
3. Rancang observer A dengan filter Kalman (gain tunak) berdasarkan variansi derau potensiometer dan model. Bandingkan dengan observer hasil pole placement.
4. Gunakan observer orde penuh dengan dua keluaran (posisi dan kecepatan) sekaligus. Bagaimana memilih gain L berukuran 2×2?

Daftar Pustaka

[1] K. Ogata, *Discrete-Time Control Systems*, edisi ke-2. Upper Saddle River, NJ: Prentice Hall, 1995.

[2] K. J. Åström dan B. Wittenmark, *Computer-Controlled Systems: Theory and Design*, edisi ke-3. Englewood Cliffs, NJ: Prentice Hall, 1997.

[3] G. F. Franklin, J. D. Powell, dan M. L. Workman, *Digital Control of Dynamic Systems*, edisi ke-3. Menlo Park, CA: Addison-Wesley, 1998.

A Kode Program Pendukung

Tab `6_state_space.ino` ditambahkan ke folder program. Program utama `11_main_state_space` menggantikan isi tab `9_main.ino`. Tab lain tetap sama seperti pada Modul 3.

Kode 1: Tab `6_state_space.ino`

```
// ============================================================================
// Tab ruang keadaan (Modul 4): model diskrit, umpan balik keadaan, observer
//
// Keadaan x = [posisi (%); kecepatan bertanda (%)], masukan u (%).
// Model diskrit (ZOH, waktu tunda diabaikan):
//   x[k+1] = A x[k] + B u[k]
//   A = [1  A12; 0  A22],  B = [B1; B2]
// Parameter model berasal dari Modul 1 (ModelK, ModelTau) dan Modul 3
// (ModelC). Gain dihitung dengan skrip desain_state_space.m.
// ============================================================================

// Parameter model (Tau dalam ms, C dalam %/ms per % kecepatan)
float ModelK = 0.85, ModelTau = 60.0, ModelC = 0.02;
float A12 = 0.0, A22 = 0.0, B1 = 0.0, B2 = 0.0;

// Pole placement kecepatan (satu keadaan): u = Nspeed*r - Kspeed*v
float Kspeed = 0.0, Nspeed = 1.0;

// Pole placement posisi (dua keadaan): u = -(Kpos*(p - r) + Kvel*v)
float Kpos = 0.5, Kvel = 0.25;

// Observer A: keluaran terukur posisi (potensiometer) -> estimasi kecepatan
float Lp1 = 0.5, Lp2 = 0.4;
float PosHat = 0.0, VelHat = 0.0;

// Observer B: keluaran terukur kecepatan (tachometer) -> estimasi posisi
float Lv1 = 0.0, Lv2 = 0.0;
float PosHatT = 0.0, VelHatT = 0.0;

void state_space_setup(unsigned int Ts) {
  float a = exp(-(float) Ts / ModelTau);
  A12 = ModelC * ModelTau * (1.0 - a);
  A22 = a;
  B1 = ModelC * ModelK * ((float) Ts - ModelTau * (1.0 - a));
  B2 = ModelK * (1.0 - a);
}

float wrap_error(float e) {
  while (e > 50.0) e -= 100.0;
  while (e < -50.0) e += 100.0;
  return e;
}

float wrap_position(float p) {
  while (p >= 100.0) p -= 100.0;
  while (p < 0.0) p += 100.0;
  return p;
}

void observer_reset(float Position, float Velocity) {
  PosHat = Position;
  VelHat = Velocity;
  PosHatT = Position;
```

```
  VelHatT = Velocity;
}

// Dipanggil sekali setiap waktu cacah, setelah masukan u[k] diberikan.
// Menghasilkan estimasi keadaan untuk cacah berikutnya, x_hat[k+1].
void observer_update(float PositionMeasured, float VelocityMeasured, float U) {
  // Observer A: koreksi berdasarkan error posisi (dibatasi -50 hingga 50)
  float Innovation = wrap_error(PositionMeasured - PosHat);
  float NextPos = PosHat + A12 * VelHat + B1 * U + Lp1 * Innovation;
  float NextVel = A22 * VelHat + B2 * U + Lp2 * Innovation;
  PosHat = wrap_position(NextPos);
  VelHat = NextVel;

  // Observer B: koreksi berdasarkan error kecepatan
  float InnovationT = VelocityMeasured - VelHatT;
  float NextPosT = PosHatT + A12 * VelHatT + B1 * U + Lv1 * InnovationT;
  float NextVelT = A22 * VelHatT + B2 * U + Lv2 * InnovationT;
  PosHatT = wrap_position(NextPosT);
  VelHatT = NextVelT;
}
```

Kode 2: Program utama `11_main_state_space`

```
// ============================================================================
// Program utama Modul 4: pole placement dan observer
// Ganti isi tab 9_main.ino dengan program ini. Tab 6_state_space.ino harus
// sudah ditambahkan.
//
// Mode = 0 : kalang terbuka. Sequence[] berisi masukan motor (%). Digunakan
//            untuk menguji kedua observer.
// Mode = 1 : pole placement kecepatan. Sequence[] berisi set point kecepatan
//            (%, positif saja).
// Mode = 2 : pole placement posisi. Sequence[] berisi set point posisi (%).
//            Kecepatan umpan balik dari observer (UseObserver = true) atau
//            dari tachometer bertanda (UseObserver = false). Setelah urutan
//            selesai, set point terakhir dipertahankan.
// ============================================================================

unsigned long Time, StartTime, LastTime, StepStart;
unsigned int TimeSampling = 10, DeltaTime;   // waktu cacah (ms)
unsigned int HoldTime = 3000;                // lama tiap nilai Sequence (ms)
byte Mode = 0;
bool UseObserver = true;
float UMax = 40.0;                           // batas masukan pada Mode 2 (%)

float Sequence[] = {0, 30, 0, -30, 0, 60, 0, -60, 0};
const byte SequenceLength = sizeof(Sequence) / sizeof(Sequence[0]);

byte Index = 0;
float Reference = 0.0, PositionMeasured = 0.0, Tacho = 0.0;
float SpeedSign = 1.0, VelocitySigned = 0.0, U = 0.0, Motor = 0.0;
bool Finished = false;

void setup() {
  general_setup();
  sensor_setup();
  actuator_setup();

  // Parameter model dan gain hasil desain_state_space.m
  ModelK = 0.85;  ModelTau = 60.0;  ModelC = 0.02;
  Kspeed = 0.0;   Nspeed = 1.0;
```

```
  Kpos = 0.5;     Kvel = 0.25;
  Lp1 = 0.5;      Lp2 = 0.4;
  Lv1 = 0.0;      Lv2 = 0.0;

  state_space_setup(TimeSampling);
  observer_reset(smooth_position_percentage(), 0.0);

  Serial.println("Time\tDelta\tRef\tPos\tPosHat\tPosHatT\tVelT\tVelHat\tU");
  StartTime = millis();
  LastTime = StartTime;
  StepStart = StartTime;
}

void loop() {
  Time = millis();
  if (Time - LastTime >= TimeSampling) {
    DeltaTime = Time - LastTime;
    LastTime = Time;

    Reference = Sequence[Index];
    PositionMeasured = smooth_position_percentage();
    Tacho = smooth_velocity_percentage();

    // Tachometer tidak membedakan arah: tanda diambil dari arah putar
    // terakhir yang diperintahkan ke driver (variabel Direction).
    if (Direction != 0) SpeedSign = (float) Direction;
    VelocitySigned = SpeedSign * Tacho;

    if (Finished && Mode != 2) {
      U = 0.0;
    }
    else if (Mode == 0) {
      U = Reference;
    }
    else if (Mode == 1) {
      if (Reference <= 0.0) U = 0.0;
      else U = Nspeed * Reference - Kspeed * Tacho;
      if (U > 100.0) U = 100.0;
      if (U < 0.0) U = 0.0;
    }
    else {
      float Velocity = UseObserver ? VelHat : VelocitySigned;
      U = -(Kpos * wrap_error(PositionMeasured - Reference) + Kvel * Velocity);
      if (U > UMax) U = UMax;
      if (U < -UMax) U = -UMax;
    }

    Motor = run_motor(compensation(U));
    if (Motor == 0.0 && U != 0.0) U = 0.0;   // satu cacah pengereman saat arah
    berbalik
    observer_update(PositionMeasured, VelocitySigned, U);

    Serial.print(Time - StartTime);
    Serial.print("\t");
    Serial.print(DeltaTime);
    Serial.print("\t");
    Serial.print(Reference);
    Serial.print("\t");
    Serial.print(PositionMeasured);
    Serial.print("\t");
    Serial.print(PosHat);
```

```
    Serial.print("\t");
    Serial.print(PosHatT);
    Serial.print("\t");
    Serial.print(VelocitySigned);
    Serial.print("\t");
    Serial.print(VelHat);
    Serial.print("\t");
    Serial.println(U);

    if (!Finished && Time - StepStart >= HoldTime) {
      StepStart = Time;
      if (Index < SequenceLength - 1) Index++;
      else Finished = true;
    }
  }
}
```

Kode 3: Skrip MATLAB `desain_state_space.m`

```
%% desain_state_space.m
% Desain pole placement dan observer untuk kit praktikum SKD (Modul 4).
%
% Keadaan x = [posisi (%); kecepatan bertanda (%)], masukan u (%).
% Model diskrit (ZOH, waktu tunda diabaikan pada desain):
%   x[k+1] = A x[k] + B u[k]
%   A = [1, C*Tau*(1-a); 0, a],  B = [C*K*(Ts - Tau*(1-a)); K*(1-a)],  a = exp(-Ts/Tau)
%
% Skrip ini:
%   1. menghitung A dan B, keterkendalian, dan keteramatan,
%   2. menghitung gain pole placement kecepatan (1 keadaan) dan posisi (2 keadaan),
%   3. menghitung gain observer kecepatan dari potensiometer (observer A)
%      dan estimator posisi dari tachometer (observer B),
%   4. mensimulasikan sistem dengan waktu tunda, saturasi, dan derau sensor,
%   5. menampilkan baris program untuk disalin ke 11_main_state_space.ino.

clc; clear; close all;

%% 1. Parameter model (waktu dalam ms)
K     = 0.85;     % penguatan (Modul 1)
Tau   = 60;       % konstanta waktu (ms) (Modul 1)
Theta = 30;       % waktu tunda (ms) (Modul 1), hanya untuk simulasi
C     = 0.02;     % konversi kecepatan ke laju posisi (%/ms per %) (Modul 3)
Ts    = 10;       % waktu cacah (ms)
UMax  = 40;       % batas masukan kontrol posisi (%)

%% 2. Spesifikasi desain
TauSpeed = 30;    % konstanta waktu kalang tertutup kontrol kecepatan (ms)
Zeta     = 0.8;   % rasio redaman kontrol posisi
TsPos    = 400;   % settling time 2% kontrol posisi (ms)
ZetaObs  = 0.8;   % rasio redaman observer A
TsObs    = 100;   % settling time observer A (ms), lebih cepat dari kontrol
TauObsV  = 20;    % konstanta waktu koreksi kecepatan observer B (ms)

%% 3. Model diskrit, keterkendalian, dan keteramatan
a = exp(-Ts/Tau);
A = [1, C*Tau*(1 - a); 0, a];
B = [C*K*(Ts - Tau*(1 - a)); K*(1 - a)];
Cpos = [1 0];     % keluaran potensiometer
Cvel = [0 1];     % keluaran tachometer
Wc   = [B, A*B];
```

```
WoP  = [Cpos; Cpos*A];
WoV  = [Cvel; Cvel*A];
fprintf('A = [%.5f %.5f; %.5f %.5f],  B = [%.6f; %.6f]\n', A', B);
fprintf('Rank matriks keterkendalian           : %d\n', rank(Wc));
fprintf('Rank matriks keteramatan (posisi)     : %d\n', rank(WoP));
fprintf('Rank matriks keteramatan (kecepatan)  : %d\n', rank(WoV));

%% 4. Pole placement kecepatan: u = Nspeed*r - Kspeed*v
zSpeed = exp(-Ts/TauSpeed);
Kspeed = (a - zSpeed) / B(2);
Nspeed = (1 - zSpeed) / B(2);

%% 5. Pole placement posisi: u = -(Kpos*(p - r) + Kvel*v)
zCtrl = kutub_diskrit(Zeta, TsPos, Ts);
Kf = ackermann(A, B, zCtrl);
Kpos = Kf(1); Kvel = Kf(2);

%% 6. Observer A (posisi terukur) dan observer B (kecepatan terukur)
zObs = kutub_diskrit(ZetaObs, TsObs, Ts);
Lp = ackermann(A', Cpos', zObs)';      % dualitas: L = acker(A', C', p)'
zVel = exp(-Ts/TauObsV);
Lv = [A(1,2); a - zVel];               % Lv1 = A12: posisi = integral kecepatan
    terukur

fprintf('\nKutub kontrol posisi : %s\n', mat2str(eig(A - B*Kf).', 4));
fprintf('Kutub observer A     : %s\n', mat2str(eig(A - Lp*Cpos).', 4));
fprintf('Kutub observer B     : %s\n', mat2str(eig(A - Lv*Cvel).', 4));

fprintf('\n// ---- salin ke setup() pada 11_main_state_space.ino ----\n');
fprintf('ModelK = %.4f;  ModelTau = %.2f;  ModelC = %.5f;\n', K, Tau, C);
fprintf('Kspeed = %.4f;  Nspeed = %.4f;\n', Kspeed, Nspeed);
fprintf('Kpos = %.4f;  Kvel = %.4f;\n', Kpos, Kvel);
fprintf('Lp1 = %.4f;  Lp2 = %.4f;\n', Lp(1), Lp(2));
fprintf('Lv1 = %.4f;  Lv2 = %.4f;\n', Lv(1), Lv(2));
fprintf('// TimeSampling = %g;\n', Ts);

%% 7. Simulasi
Prm = struct('K', K, 'Tau', Tau, 'Theta', Theta, 'C', C, 'Ts', Ts, 'UMax', UMax, ...
    'Kspeed', Kspeed, 'Nspeed', Nspeed, 'Kpos', Kpos, 'Kvel', Kvel, 'Lp', Lp, 'Lv',
    Lv, ...
    'A', A, 'B', B, 'TachoMin', 3, 'Derau', 0.5);

% (a) Kalang terbuka: uji kedua observer
S = simulasi(Prm, 0, [0 30 0 -30 0 60 0 -60 0], 3000, true);
figure('Name', 'Observer (kalang terbuka)');
subplot(2,1,1); plot(S.t, S.v, 'k', S.t, S.vT, 'b.', S.t, S.vHat, 'r'); grid on;
legend('Kecepatan sebenarnya', 'Tachometer bertanda', 'Estimasi observer A');
ylabel('Kecepatan (%)');
subplot(2,1,2); plot(S.t, S.pTrue, 'k', S.t, S.pHatT, 'r'); grid on;
legend('Posisi sebenarnya (tanpa batas)', 'Estimasi observer B');
xlabel('Waktu (ms)'); ylabel('Posisi (%)');

% (b) Pole placement kecepatan
S = simulasi(Prm, 1, [0 40 70 30 0], 1500, true);
figure('Name', 'Pole placement kecepatan');
stairs(S.t, S.r, 'k--'); hold on; plot(S.t, S.v, 'b'); grid on;
xlabel('Waktu (ms)'); ylabel('Kecepatan (%)'); legend('Set point', 'Kecepatan');

% (c) Pole placement posisi: observer vs tachometer bertanda
Ref = [50 20 50 90 20 50];
S1 = simulasi(Prm, 2, Ref, 2000, true);
```

```
S2 = simulasi(Prm, 2, Ref, 2000, false);
figure('Name', 'Pole placement posisi');
stairs(S1.t, S1.r, 'k--'); hold on;
plot(S1.t, S1.p, 'b', S2.t, S2.p, 'r'); grid on;
xlabel('Waktu (ms)'); ylabel('Posisi (%)');
legend('Set point', 'Umpan balik observer', 'Umpan balik tachometer');

%% Fungsi bantu
function z = kutub_diskrit(Zeta, TSettle, Ts)
    % Pasangan kutub diskrit dari rasio redaman dan settling time 2%
    wn = 4 / (Zeta * TSettle);
    s  = -Zeta*wn + 1i*wn*sqrt(1 - Zeta^2);
    z  = exp([s; conj(s)] * Ts);
end

function Kg = ackermann(A, B, Kutub)
    % Rumus Ackermann untuk sistem orde dua
    al  = real(poly(Kutub));              % [1 alpha1 alpha0]
    Phi = A^2 + al(2)*A + al(3)*eye(2);
    Kg  = [0 1] / [B, A*B] * Phi;
end

function S = simulasi(Prm, Mode, Seq, Hold, PakaiObserver)
    % Plant kontinu (langkah 1 ms) dengan waktu tunda, saturasi, dan derau.
    Ts = Prm.Ts; d = round(Prm.Theta); N = Hold * numel(Seq);
    ubuf = zeros(N + d + 1, 1);
    v = 0; p = 50; u = 0; Arah = 1;
    xA = [50; 0]; xB = [50; 0];
    nk = floor((N - 1)/Ts) + 1;
    S = struct('t', zeros(nk,1), 'r', zeros(nk,1), 'v', zeros(nk,1), 'vT', zeros(nk
    ,1), ...
        'vHat', zeros(nk,1), 'p', zeros(nk,1), 'pTrue', zeros(nk,1), 'pHatT', zeros(
    nk,1));
    i = 0;
    for k = 0:N-1
        if mod(k, Ts) == 0
            r = Seq(min(numel(Seq), floor(k/Hold) + 1));
            pm = mod(p + 0.1*randn, 100);                    % potensiometer
            vt = abs(v); if vt < Prm.TachoMin, vt = 0; end   % tachometer
            vt = max(0, vt + Prm.Derau*randn*(vt > 0));
            if u ~= 0, Arah = sign(u); end
            vs = Arah * vt;
            switch Mode
                case 0
                    u = r;
                case 1
                    if r <= 0, u = 0; else, u = Prm.Nspeed*r - Prm.Kspeed*vt; end
                    u = min(100, max(0, u));
                otherwise
                    if PakaiObserver, vf = xA(2); else, vf = vs; end
                    u = -(Prm.Kpos*bungkus(pm - r) + Prm.Kvel*vf);
                    u = min(Prm.UMax, max(-Prm.UMax, u));
            end
            % observer A (posisi terukur) dan observer B (kecepatan terukur)
            xA = Prm.A*xA + Prm.B*u + Prm.Lp*bungkus(pm - xA(1)); xA(1) = mod(xA(1),
     100);
            xB = Prm.A*xB + Prm.B*u + Prm.Lv*(vs - xB(2));
            i = i + 1;
            S.t(i) = k; S.r(i) = r; S.v(i) = v; S.vT(i) = vs; S.vHat(i) = xA(2);
            S.p(i) = pm; S.pTrue(i) = p; S.pHatT(i) = xB(1);
        end
```

```
        ubuf(k + 1 + d) = u;
        v = v + (Prm.K*ubuf(k + 1) - v) / Prm.Tau;
        p = p + Prm.C * v;
    end
end

function e = bungkus(e)
    % Membatasi error posisi pada rentang -50 hingga 50
    e = mod(e + 50, 100) - 50;
end
```

Kode 4: Skrip MATLAB `analisis_observer.m`

```
%% analisis_observer.m
% Analisis data percobaan observer (Modul 4) dari program 11_main_state_space.
% Kolom data: Time | Delta | Ref | Pos | PosHat | PosHatT | VelT | VelHat | U

clc; clear; close all;

NamaFile = 'data_observer.txt';
C  = 0.02;          % konversi kecepatan ke laju posisi (%/ms per %), dari Modul 3
VelMin = 5;         % batas kecepatan (%) untuk membandingkan estimasi dengan
    tachometer

Data = readmatrix(NamaFile);
Data = Data(all(~isnan(Data(:, 1:9)), 2), :);
t      = Data(:,1) - Data(1,1);
Ref    = Data(:,3);
Pos    = Data(:,4);
PosHat = Data(:,5);
PosHatT= Data(:,6);
VelT   = Data(:,7);
VelHat = Data(:,8);
U      = Data(:,9);

% Posisi tanpa batas 0-100 (unwrap dengan periode 100%)
PosU     = buka(Pos);
PosHatTU = buka(PosHatT);

% Kecepatan dari turunan numerik posisi (pembanding sederhana)
VelDiff = [0; diff(PosU) ./ diff(t)] / C;

%% Observer A: estimasi kecepatan dari potensiometer
Bergerak = abs(VelT) > VelMin;
RMSE_A = sqrt(mean((VelHat(Bergerak) - VelT(Bergerak)).^2));
RMSE_D = sqrt(mean((VelDiff(Bergerak) - VelT(Bergerak)).^2));
fprintf('RMSE kecepatan observer A terhadap tachometer : %.2f %%\n', RMSE_A);
fprintf('RMSE kecepatan turunan numerik posisi        : %.2f %%\n', RMSE_D);

figure('Name', 'Observer A');
plot(t, VelT, 'b.', t, VelDiff, 'Color', [0.6 0.6 0.6]); hold on;
plot(t, VelHat, 'r', 'LineWidth', 1.2); grid on;
xlabel('Waktu (ms)'); ylabel('Kecepatan (%)');
legend('Tachometer bertanda', 'Turunan numerik posisi', 'Estimasi observer A');

%% Observer B: estimasi posisi dari tachometer
ErrB = PosHatTU - PosU;
fprintf('Error posisi observer B: awal %.2f %%, akhir %.2f %%, maksimum %.2f %%\n',
    ...
    ErrB(1), ErrB(end), max(abs(ErrB)));

```

```
figure('Name', 'Observer B');
subplot(2,1,1); plot(t, PosU, 'k', t, PosHatTU, 'r'); grid on;
ylabel('Posisi (%, tanpa batas)'); legend('Potensiometer', 'Estimasi observer B');
subplot(2,1,2); plot(t, ErrB, 'r'); grid on;
xlabel('Waktu (ms)'); ylabel('Error posisi (%)');

%% Fungsi bantu
function y = buka(x)
    % unwrap posisi dengan periode 100%
    y = x;
    for k = 2:numel(x)
        dx = x(k) - x(k-1);
        dx = mod(dx + 50, 100) - 50;
        y(k) = y(k-1) + dx;
    end
end
```